DEEP LEARNING

Generative Modelling

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

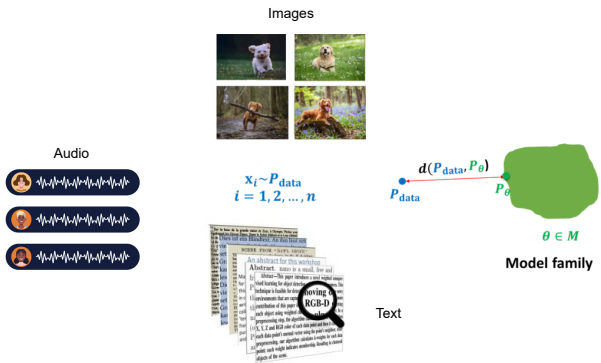
Table of Contents

Unsupervised Learning

- Unsupervised learning is a type of machine learning that learns from data **without** human supervision, i.e., without target labels.
- Some goals and applications:
  - Learning representations (e.g., self-supervised learning)
  - Data Compression (e.g., Autoencoders)
  - Finding hidden structures in data (e.g., clustering)
  - Generating new examples (e.g., Variational Autoencoders and more today)
  - Many more...
- Why unsupervised learning?
  - Unlabelled data is way cheaper than labelled data
  - Unlabelled data is found in abundance

Learning A Generative Model

- We are given a set of training examples x_i (e.g., images, audio signals or text) that are drawn from an arbitrary data distribution p_{data}.



- Goal in generative modelling is to learn the underlying (unknown) probability distribution, denoted by p_{data}.

Learning A Generative Model

- Why do we want to learn a generative model?
  - **Data generation**: We can get new samples (e.g., new dog, new audio track, new poem) by sampling from the learned distribution.
  - **Density estimation**: Given a datapoint, we can tell if it comes from the data distribution by evaluating $p(x)$. If it comes from the distribution, then $p(x)$ is high, otherwise not (e.g., anomaly detection).
  - **Learning representation**: We can a get compact representation of our training data.
- Our goal is to learn a parameterized model $p_\theta(x)$ such that: $p_\theta(x) \approx p_{\text{data}}(x)$.
- In generative modelling we need to answer two questions:
  - How to represent $p_\theta(x)$?
  - How to learn it?

Applications of Generative Models

Creative Writing

Prompt to AI:

> *"Create a vivid D&D campaign hook for a group of Level 5 adventurers. The setting is a cursed coastal town where the sea has stopped moving."*

Title: "The Still Sea of Varrow's Reach"

The sea should never be silent.

But in the mist-cloaked town of Varrow's Reach, the tides have frozen mid-crash, waves hanging like glass sculptures along the jagged cliffs. Ships rest in the harbor as if caught in amber. Even the gulls have vanished.

The townsfolk speak in whispers of a **forgotten god** beneath the waves, once worshipped, now betrayed. They say its eye has opened.

Fishermen report strange lights under the unmoving sea. The lighthouse burns day and night, but no ships dare leave.

You've been summoned by the town's last sane cleric, who believes a **ritual was interrupted**—and that time itself has stopped flowing in the waters.

He warns: if the sea doesn't move soon, **something else will**.

Adventure awaits in the drowned catacombs, aboard ghost ships trapped in time, and within dreams whispered by the deep.

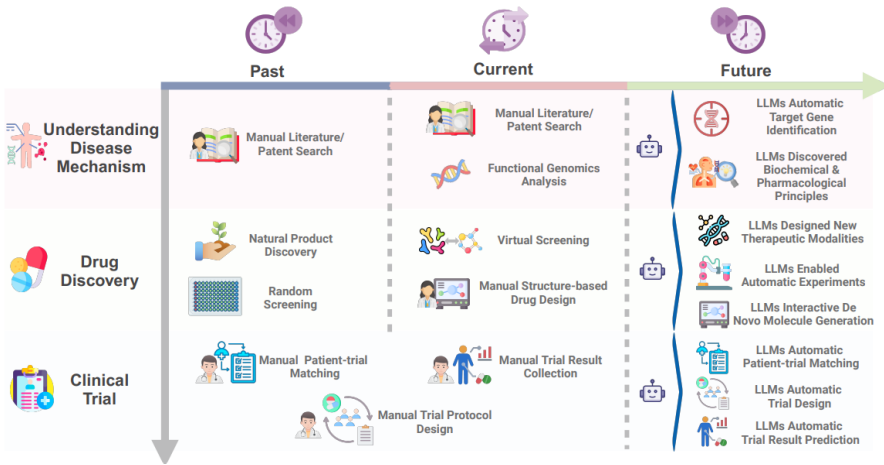
Will your party break the curse—or become part of the stillness?

Creating Artworks

Prompt: Create a vivid D&D campaign hook for a group of Level 5 adventurers. The setting is a cursed coastal town where the sea has stopped moving.

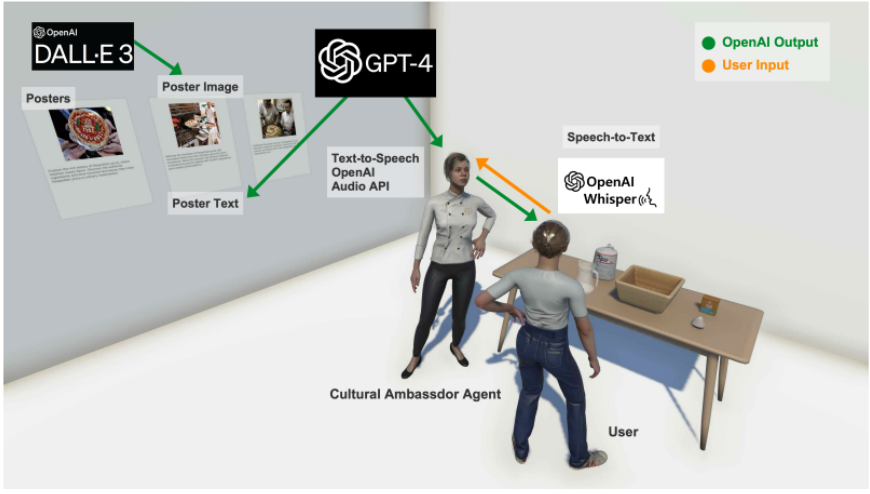
[0] https://playground.bfl.ai/image/generate

Drug Discovery and Development



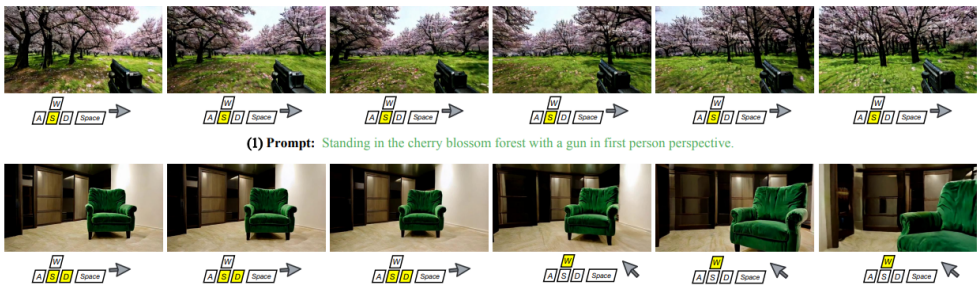
[0] https://arxiv.org/pdf/2409.04481

Virtual Reality



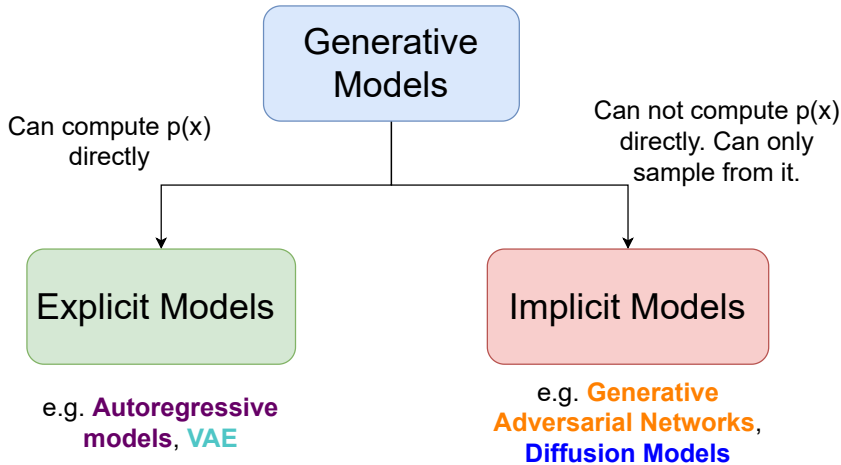
[0] https://arxiv.org/pdf/2411.18438

Creating New Videogames



[0] https://yujiwen.github.io/gamefactory/

Taxonomy of Generative Models



Learning A Generative Model

- **Goal**: To estimate p_{data}.
- We introduce a parameterized model $p_\theta(x)$ with the goal of learning the parameters θ such that $p_\theta(x) \approx p_{\text{data}}(x)$.
- We do not know the analytical form of $p_{\text{data}}(x)$. Instead, we just have access to datapoints drawn from the unknown distribution: $x_i \sim p_{\text{data}}(x)$.
- This is challenging because:
  - We have access to limited data. It only provides us a rough approximation of the true underlying distribution.
  - Computational reasons. Sometimes the distribution is not *tractable* (we will see later what that means).

Autoregressive Models

Maximum Likelihood Estimation

- Maximum likelihood estimation (MLE): A training mechanism for adjusting the model parameters to maximize the probability of the observed training data.
- Given a dataset $\{\mathbf{x}^{(i)}\}_{i=1}^{n}$,find the parameters θ by solving the optimization problem:

$$\arg\min_{\theta} \text{loss}(\theta, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \cdots, \mathbf{x}^{(n)}) = \frac{1}{n} \sum_{i=1}^{n} \overbrace{-\log p_\theta(\mathbf{x}^{(i)})}^{\text{negative log-likelihood}} .$$

- **Intuitively**, the above optimization is trying to make $p_\theta(\mathbf{x})$ assign high probability to instances sampled from p_{data}.
- Formally, solving the maximum likelihood problem will yield parameters that generate the data.
- How do we solve this optimization problem?
  - Using Gradient Descent (or SGD if we are dealing with neural networks and large datasets).

Autoregressive Models

- Any structured data can be transformed into a sequence!
- **Autoregressive models** (AR) are a type of ML models that predicts the next value in a sequence based on the previous values in that sequence.
- We have seen AR models in the context of natural language processing:
  - Recurrent Neural Networks
  - Language Modelling with Transformers
- Let's say we have a sequence of d variables or features: $\mathbf{x} = (x_1, x_2, \cdots, x_d)$.
- **Generative models** are after finding the joint probability distribution:

$$p(\mathbf{x}) = P(x_1, x_2, \cdots, x_d)$$

- AR models use chain-rule to write it as a product of the conditional probabilities:

$$p(x_1, \cdots, x_d) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_d \mid, x_1, \cdots, x_{d-1})$$

[0] `https://www.getpeech.com/blog/autoregressive-models-world`

Extending MLE to Autoregressive Models

- For example, in the case of MNIST digits, each image is $28 \times 28 = 784$ pixels. We can write it as:

$$p(x_1, \cdots, x_{784}) = p(x_1)p(x_2 \mid x_1)p(x_3 \mid x_1, x_2) \cdots p(x_d \mid, x_1, \cdots, x_{783})$$

- We can write more compactly as:

$$p_\theta(\mathbf{x}) = \prod_{j=1}^{d} p_\theta(x_j \mid x_{1:j-1}; \theta)$$

- Given a training dataset $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n$ and learnable parameters θ, the goal of a AR model is to maximize the likelihood. In other words, use MLE to estimate the parameters θ.

MLE using Stochastic Gradient Descent

- The likelihood for an AR model is given as:

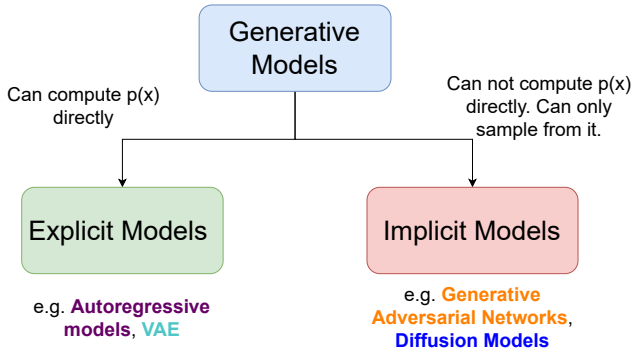
$$L(\theta, \mathcal{D}) = \prod_{i=1}^{n} p_\theta(\mathbf{x}^{(i)}) = \prod_{i=1}^{n} \prod_{j=1}^{d} p_\theta(x_j^{(i)} \mid x_{1:j-1}^{(i)}; \theta)$$

- Goal: find θ such that $\arg\max_\theta L(\theta, \mathcal{D})$. Or equivalently: $\arg\min_\theta - \log L(\theta, \mathcal{D})$ (by taking logarithm, products change to sums).

$$\ell(\theta) = \log L(\theta, \mathcal{D}) = \sum_{i=1}^{n} \sum_{j=1}^{d} p_\theta(x_j^{(i)} \mid x_{1:j-1}^{(i)}; \theta)$$

- If p_θ is a neural network, then we can train with backpropagation and SGD, as we have done for any regular neural network.

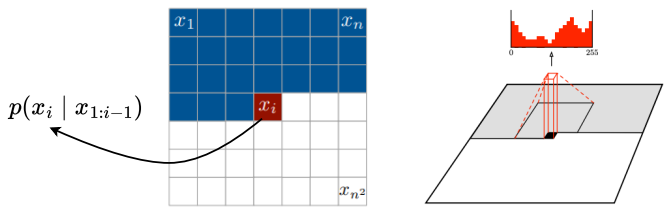
Recap: Autoregressive Models



- We said here that AR models falls under "explicit models". Why?
- A: Because we are **directly** estimating the parameterized distribution $p_\theta(x)$ using MLE.

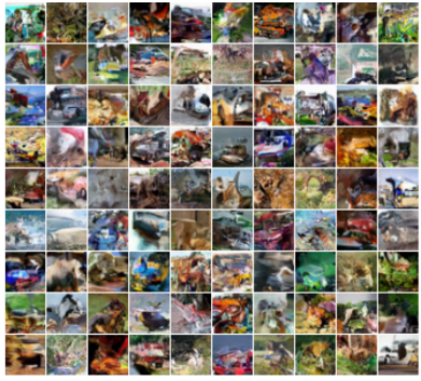
PixelCNN

- The model is trained autoregressively to predict the pixel value at the i^{th} location by only looking at the pixels to the left and above. Input: (B, H, W, C); Output: (B, H, W, C, 256).

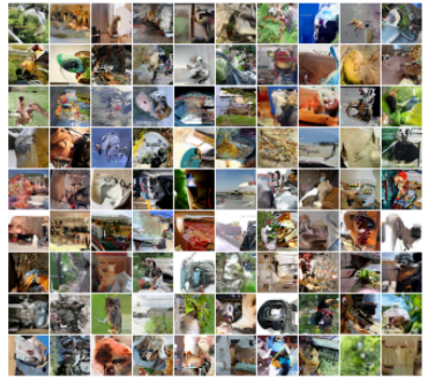


- Uses masked convolution (think of masked self-attention) to achieve this.
- The training can be parallelized because of the convolution operation and the knowledge of the entire set of pixels in the training set (similar to Transformers).
- However, at inference time the generation is sequential, and hence slow!

PixelCNN



CIFAR-10

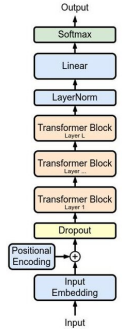


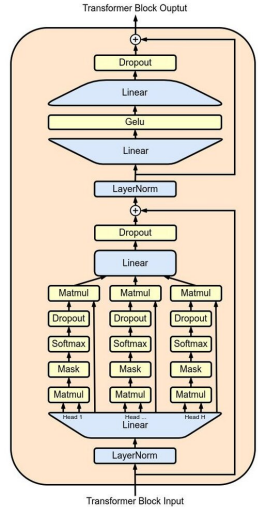
ImageNet 32

- Samples from models trained on CIFAR-10 (left) and ImageNet 32x32 (right) images.
- They capture some local structures, but still far from realistics generations.

Generative Pre-trained Transformer (GPT)

- Autoregressive language model using transformers to generate human-like text.
- It uses Decoder-only transformer architecture (self-attention + feedforward layers).
- Trained using next token prediction on large-scale text (hence unsupervised learning).
- After pre-training it is fine-tuned on supervised tasks that has labels (e.g., QA, summarization).





GPT-2

- Same as GPT-1 in terms of training objective and architecture.
- Improvements in terms of:
  - Bigger model (1.5B parameters)
  - Trained on 40GB WebText
- They showed that with scale (both model size and amount of data) language models can perform downstream tasks without fine-tuning.

"I'm not the cleverest man in the world, but like they say in French: **Je ne suis pas un imbecile [I'm not a fool].**

In a now-deleted post from Aug. 16, Soheil Eid, Tory candidate in the riding of Joliette, wrote in French: "**Mentez mentez, il en restera toujours quelque chose**," which translates as, "**Lie lie and something will always remain.**"

"I hate the word '**perfume**,'" Burr says. 'It's somewhat better in French: '**parfum**.'

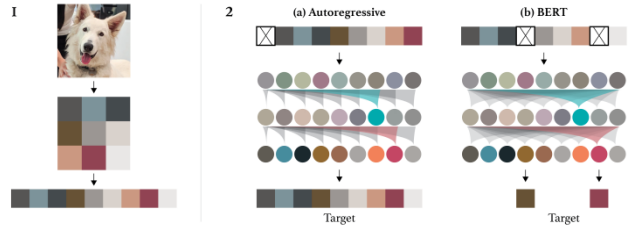
If listened carefully at 29:55, a conversation can be heard between two guys in French: "**-Comment on fait pour aller de l'autre coté? -Quel autre coté?**", which means "**- How do you get to the other side? - What side?**".

If this sounds like a bit of a stretch, consider this question in French: **As-tu aller au cinéma?**, or **Did you go to the movies?**, which literally translates as Have-you to go to movies/theater?

"**Brevet Sans Garantie Du Gouvernement**", translated to English: "**Patented without government warranty**".

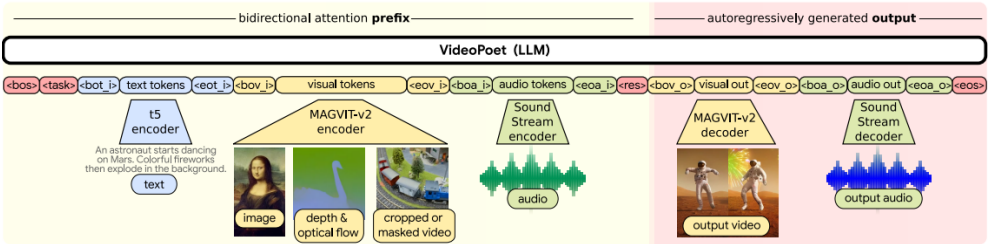
Table 1. Examples of naturally occurring demonstrations of English to French and French to English translation found throughout the WebText training set.

Image GPT (iGPT)



- **Key idea**: Apply GPT-like training to images.
- First, pre-process raw images by resizing to a low resolution and reshaping into a 1D sequence (details are not important).
- Use the pre-training objectives: (i) auto-regressive next pixel prediction (same as GPT), or (ii) masked pixel prediction (which is mask out some pixels and ask the model to predict those masked pixels).
- The final goal here is not to generate new pixels, but to learn good representations.

VideoPoet



- **Key idea**: Incorporate a mixture of multimodal generative objectives within an autoregressive Transformer framework. Meaning:
  - Instead of just decoding text, decode everything (images, videos, audio).
- This means, we can generate anything-from-anything (e.g., text-to-video).

VideoPoet Generations

Figure 5: **10-Second long video generation example.** By predicting 1-second video segments from an initial 1-second clip, VideoPoet can iteratively generate videos of extended lengths.

Animated from historical photo

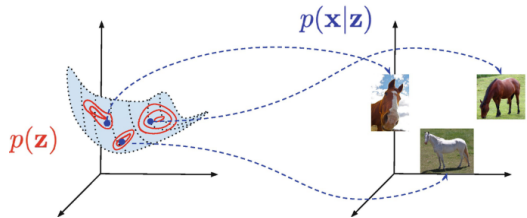
Animated from painting

Figure 6: **Examples of videos animated from still images plus text prompts tailored to each initial image.**

Latent Variable Models

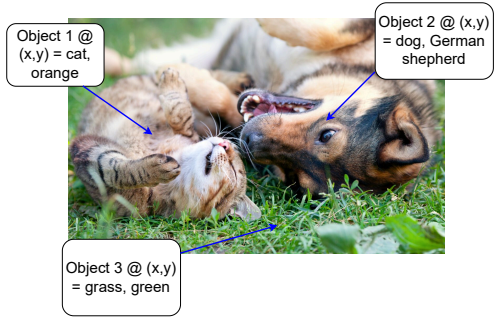
Latent Variable Models

- Latent variable models that assume observed data is generated from hidden (latent) variables.
- These latent variables capture underlying patterns or factors not directly visible.
- Examples:
  - In face images: lighting, emotion, identity may be latent variables.
  - In text: topics or sentiment may be latent.
  - In biology: gene expression patterns may be latent.
- **Key intuition**: Think of latent variables z as the "invisible causes" (or factors) behind the data x you see. Latent variable models explicitly model these factors.



Why Latent Variable Models?

- The underlying assumption is that lower-dimensional representation of data is often possible. In the above below, the image can be described by a lower dimensional representations: (location, object class, attributes).



- If we could learn this lower-dimensional representation z, then we could sample from the distribution $p(x \mid z)$ (by picking a new z) to get a new image x.
- **Challenge**: Very difficult to handcraft z or its distribution $p_Z(z)$.

Latent Variable Model

- z is latent (or unobserved) variable and x is observed variable.
- Sampling a new example means:

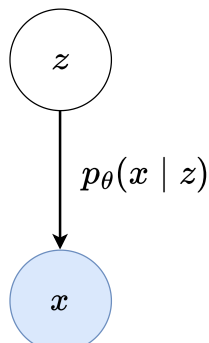
$$z \sim p_Z(z)$$
$$x \sim p_\theta(x \mid z)$$

- Evaluating likelihood means:

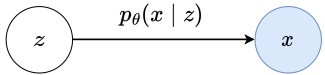
$$p_\theta(x) = \sum_z p_Z(z) p_\theta(x \mid z)$$

- Training means maximizing the likelihood:

$$\max_\theta \sum_i \log p_\theta(x^{(i)}) = \sum_i \log \sum_z p_Z(z) p_\theta(x \mid z)$$



Training Latent Variable Models



- Objective function: $\max_\theta \sum_i \log p_\theta(x^{(i)}) = \sum_i \log \sum_z p_Z(z) p_\theta(x^{(i)} \mid z)$.
- Computing the objective function means getting our hands on the values of z, the values that we can't observe. So we marginalize, i.e., sum over all the values of z. What are the ways of computing the objective:
  - Option 1: if z takes on a very small number of values, then we can easily compute the objective function $\rightarrow$ tractable (easy to deal with).
  - Option 2: if z has impractical number of values (e.g., in higher dimensions), then we can't enumerate them all. In that case we could use random sampling:

$$\sum_i \log \sum_z p_Z(z) p_\theta(x \mid z) \approx \sum_i \log \frac{1}{K} \sum_{k=1}^{K} p_\theta(x^{(i)} \mid z_k^{(i)}), \quad \text{where } z_k^{(i)} \sim p_Z(z)$$

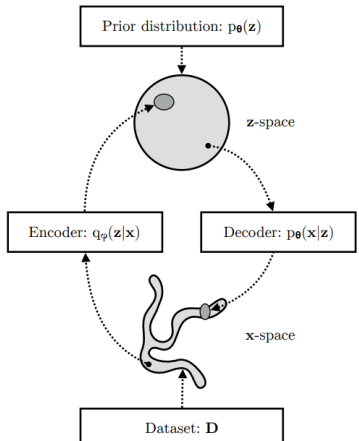
  However, it has an issue: impossible to be lucky enough that a sampled $z_k^{(i)}$ is a good match for a data point $x^{(i)}$.

Training Latent Variable Models

- Ways of computing the objective:
  - Option 3: Use importance sampling (details out of the scope of this course). Intuitively, we sample from an alternate distribution as $z_k^{(i)} \sim q(z)$, that are compatible with $x^{(i)}$.
  - How do we choose $q(z)$? How about: $q(z) = p_\theta(z \mid x^{(i)})$. It means, given $x^{(i)}$ we get a distribution over z.
  - This is nicer than before, but how do we sample from the distribution $p_\theta(z \mid x^{(i)})$ (assuming we have one)?
  - Option 4: We choose $q(z)$ to be a distribution that we can easily work with (or sample from it). For e.g., we set $q(z) = \mathcal{N}(z; \mu, \sigma^2)$ and enforce that $\mathcal{N}(z; \mu, \sigma^2) \approx p_\theta(z \mid x^{(i)})$.
  - This is the main idea behind the latent variable model Variational Autoencoders (VAE) – learn an encoder $q_\phi(z \mid x^{(i)})$ that **approximates** the true but intractable posterior $p_\theta(z \mid x^{(i)})$, where $q_\phi(z \mid x^{(i)})$ is another neural network parameterized by ϕ.
  - Unfortunately, this topic is very maths heavy, but we will skip them and go to the final results.

Variational Autoencoders (VAE)

- Run the data $x^{(i)}$ through the encoder $q_\phi(z \mid x^{(i)})$ to get a distribution over z.
- The encoder output should follow a simple prior distribution (e.g., unit Gaussian). Specifically, the encoder outputs the parameters of a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$.
- Sample z from this Gaussian distribution.
- Run z through the decoder $p_\theta(x^{(i)} \mid z)$ to get the predicted data $\hat{x}^{(i)}$.



Training a VAE

- Training a VAE means optimizing the following objective function:

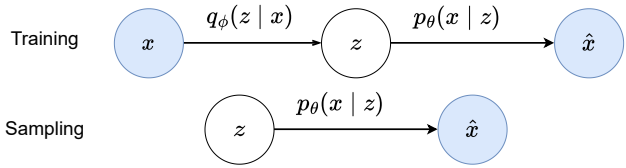
$$\arg\max_{\theta,\phi} \underbrace{\mathbb{E}_{z\sim q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{likelihood term}} - \underbrace{D_{\mathsf{KL}}(q_\phi(z \mid x)\|p(z))}_{\text{KL Divergence}}$$

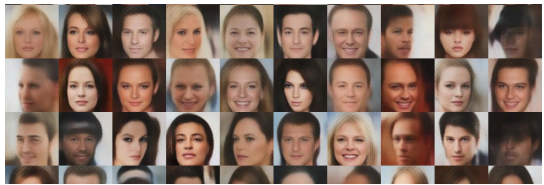
- In more practical terms, the loss function looks as follows:

$$\underbrace{\|x - \hat{x}\|_2^2}_{\text{reconstruction loss}} + \underbrace{D_{\mathsf{KL}}(q_\phi(z \mid x)\|p(z))}_{\text{regularization}}$$

- Intuitively,
  - The first term penalizes reconstruction error (which can be thought of maximizing the likelihood $p_\theta(x \mid z)$).
  - The second term is a regularization term that encourages the learned distribution $q_\phi(z \mid x)$ to be similar to the true prior distribution, which we will assume to follow a unit Gaussian distribution $p(z)$.

Sampling from VAE



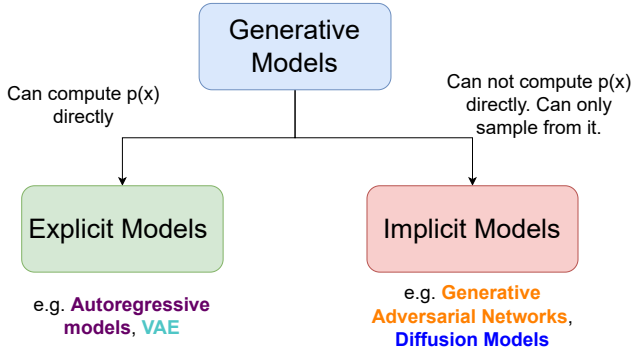


Randomly generated samples

- Sample z from the prior distribution $p(z)$ as $z \sim \mathcal{N}(0, I)$, and run it through the decoder to get new samples.

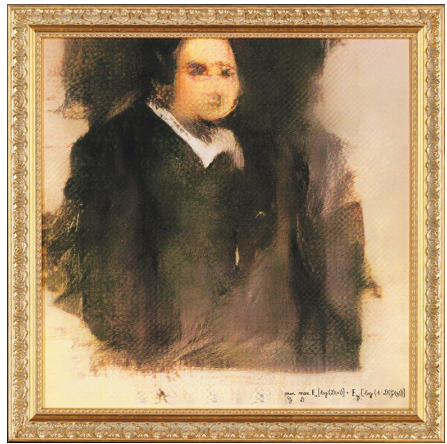
Generative Adversarial Networks

Generative Adversarial Networks



- AR and latent variable models (e.g., VAE) are both explicit models because we can either compute the likelihood of the data directly (for AR) or approximate it (for VAE).
- Generative Adversarial Networks (GANs) are implicit generative models because we can not compute the likelihood!

GAN Artwork

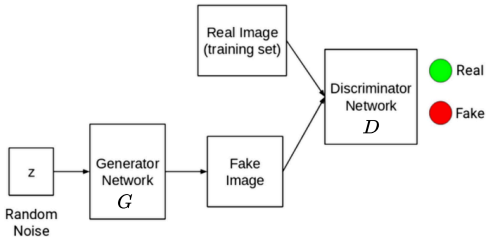


This is not a real painting!!! Believe it or not, this painting was created using GANs and was sold for more than 400k dollars in an auction! If you look closely, at the bottom right, there is the equation that governs GANs.

Implicit Models

- We are given a set of training data points from the data distribution $p_{\text{data}} : x^{(1)}, x^{(2)}, \cdots, x^{(n)}$.
- We have a model that we can draw samples from, i.e., $x = q_\phi(z)$, by drawing z from some prior distribution: $z \sim p(z)$ (e.g., Uniform or Gaussian distribution).
- Note, we do not have $p_\theta(x)$ that we can directly compute or evaluate. Rather we can only draw samples from $q_\phi(z)$.
- **Goal**: Adjust the parameters ϕ so that we can get samples that look like the samples that could have come from p_{data}.
- In other words, we are inducing a $p_\theta(x)$ through the generated samples, and hence called implicit models.

Generative Adversarial Network (GAN)

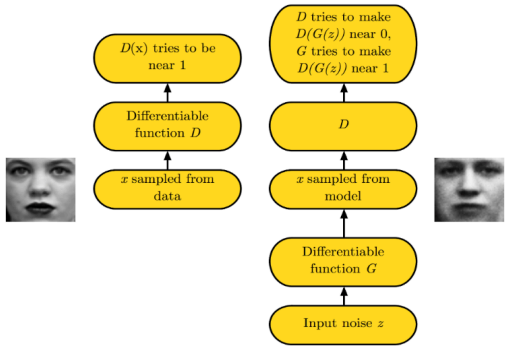


- The objective function of GAN is as follows:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}[\log(1 - D(G(z)))]$$

- Two player minimax game between a generator (G) and a discriminator (D).
- D is trying is maximize the log-likelihood of:
  - real data to be close to 1
  - fake (or generated) data to be close to 0
- G is trying to minimize the log-likelihood of its samples being classified as "fake" by D.

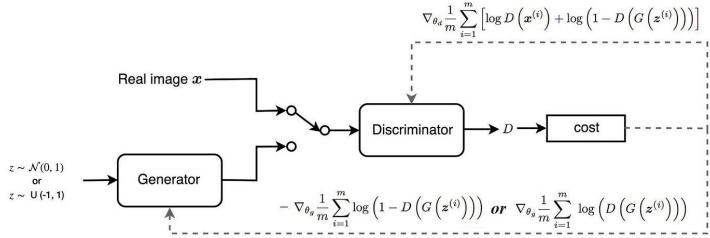
GAN



- When an equilibrium is reached (or at convergence), the D can no longer distinguish between real and fake samples.
- It means the generated samples will look like as if they have been sampled from the true data distribution p_{data}.

Training GAN

- The D and G are nothing but neural networks.
- Trained with SGD and backpropagation, like any other neural network.



- However, training GAN is not straightforward. Several follow up papers came that proposed different ideas to improve the training. We won't be covering those.
- Check for a nice demo of visualizing GAN training[1].

[1] `https://poloclub.github.io/ganlab/`

Sampling from GAN

- Once the D and G have been trained, we discard the D. We sample from the prior distribution and feed it to G to generate new samples (just like VAE).

Original GAN

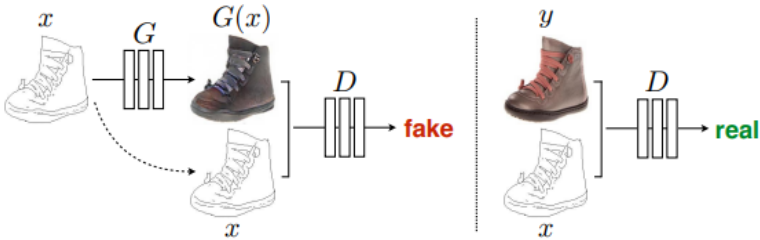
Samples from 2014

StyleGAN2

Samples from 2019

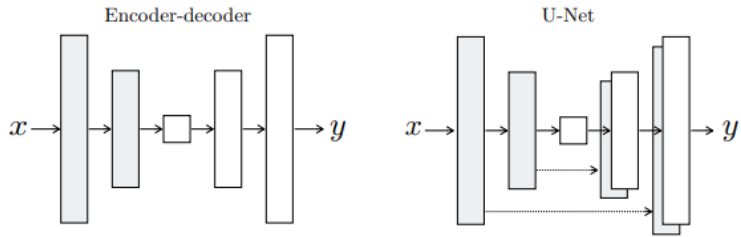
- As GAN models have gone bigger and training has improved, the quality of generated samples has massively improved the years.

Conditional GANs (pix2pix)



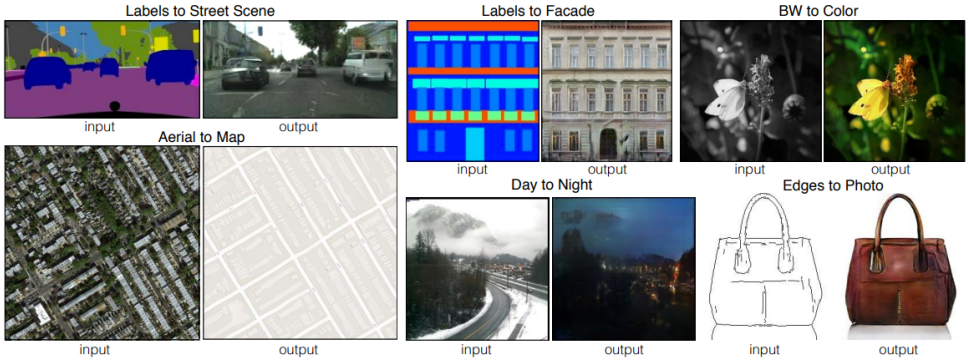
- We do not necessarily need to generate samples starting from noise. We can also give an edge map as input to the G and train the G to generate a realistic looking colored photo.
- Same as before, the D learns to classify between fake (synthesized by the G) and real {edge, photo} tuples.

Conditional GANs (pix2pix)



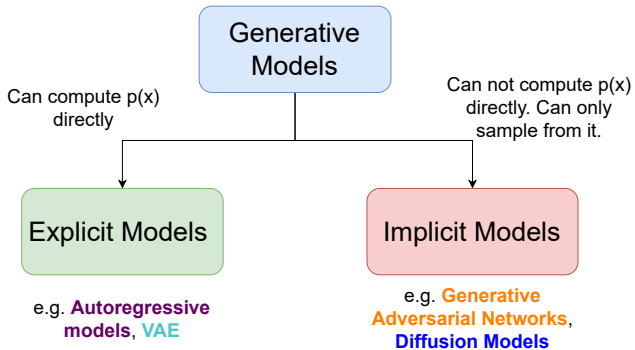
- Since we are dealing with images at both the input and the output, we will adopt an Encoder-Decoder kind of architecture (similar to ConvAE) or U-Net for the G.
- Because we want the input and output to be of same dimensions.

Conditional GANs (pix2pix)



Diffusion Models

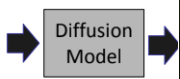
Diffusion Models



- Conceptually speaking, Diffusion Models are similar to GANs because we do not explicitly model the $p_\theta(x)$.
- Works (way) better than GANs minus the hassle.

Diffusion Models

Shot and directed by Quentin Tarantino. Cat in a business suit sitting in a crowded subway train during rush hour, commuting, sad expressions



Demo link

Diffusion Models

Beautiful, snowy Tokyo city is bustling. The camera moves through the bustling city street, following several people enjoying the beautiful snowy weather and shopping at nearby stalls. Gorgeous sakura petals are flying through the wind along with snowflakes.

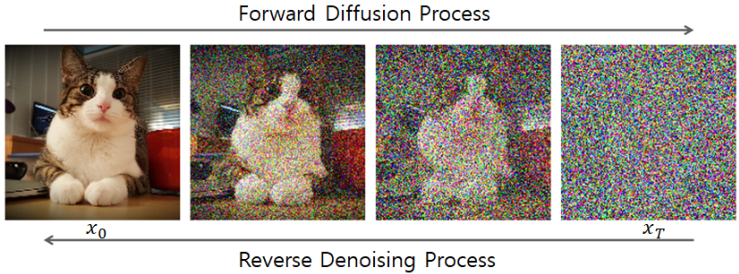
Video link

Diffusion Models

Five gray wolf pups frolicking and chasing each other around a remote gravel road, surrounded by grass. The pups run and leap, chasing each other, and nipping at each other, playing.
morePrompt: Five gray wolf pups frolicking and chasing each other around a remote gravel road, surrounded by grass. The pups run and leap, chasing each other, and nipping at each other, playing.

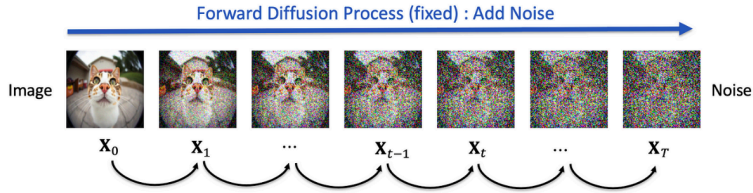
Video link

Denoising Diffusion Probabilistic Models (DDPM)



- A generative model that creates data by learning to reverse a gradual noising process.
- Inspired by thermodynamics: slowly add noise to data, then learn to reverse that process step-by-step.
- At a high-level there are two processes:
  - Gradually add Gaussian noise to data over T steps (called Forward Diffusion Process).
  - Learn a neural network to reverse the noise and recover data (called the Reverse Denoising Process).

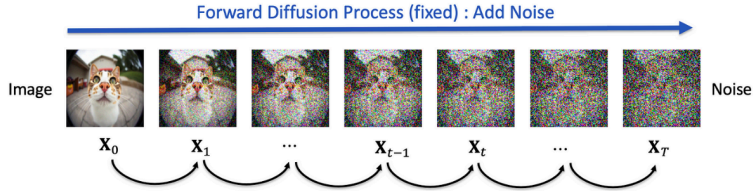
DDPM: Forward Diffusion Process



- Given a real datapoint x_0, the forward process produces a sequence $x_1, x_2, \cdots, X_T$ by adding noise at each step.
- Q: How much noise to add? The strength of the noise is determined by a variance schedule $\beta_1, \beta_2, \cdots, \beta_T$. For e.g., $\beta_1 \approx 10^{-4}$ to $\beta_T \approx 0.02$.
- The transition from x_{t-1} to x_t is defined as:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

DDPM: Forward Diffusion Process



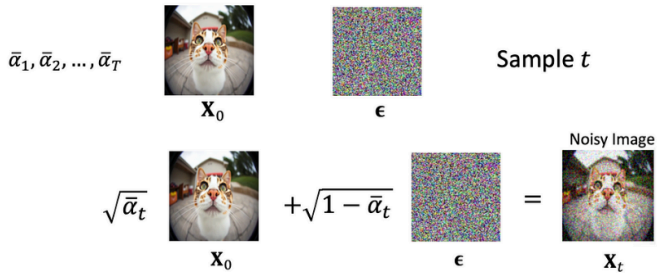
- Another way to write the same thing is:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

where,

- $\alpha_t = 1 - \beta_t$
- $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$
- $\epsilon \sim \mathcal{N}(0, I)$ is Gaussian noise

DDPM: Forward Diffusion Process

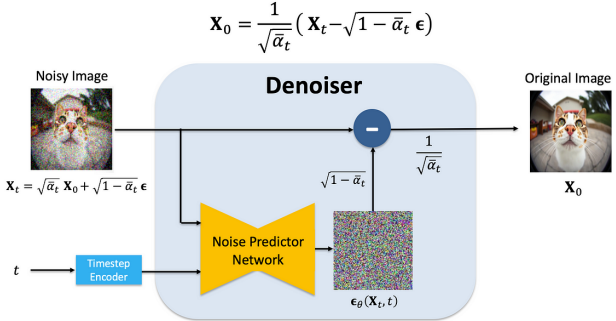


- The forward process:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

- Intuitively, the above equation is saying, given a real sample x_0 we can reach a noisy version of it at any arbitrary time-step t.

DDPM: Reverse Diffusion Process

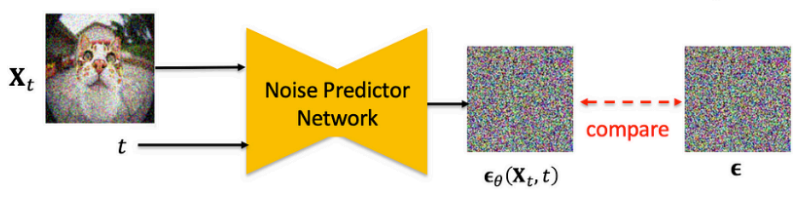


- The reverse process is parameterized by a neural network that predicts how to denoise x_t to obtain x_{t-1}.
- ϵ_θ is a neural network that predicts the noise, which could have been added to the image.
- Ignoring the scaling terms (for simplicity) we can now subtract the predicted noise $\epsilon_\theta(x_t, t)$ from x_t to get back the real image x_0.

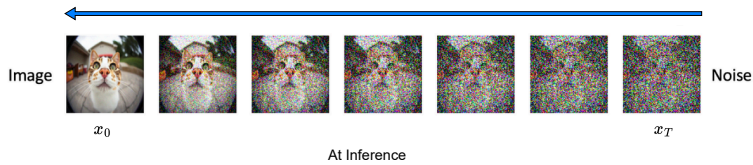
DDPM: Training

- In practice, we can not get the back the real image in one step (too difficult for the model). Rather, we sample t from a uniform distribution.
- Training objective is to minimize the MSE between predicted and true noise:

$$\mathcal{L} = \mathbb{E}_{t,x_0,\epsilon}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

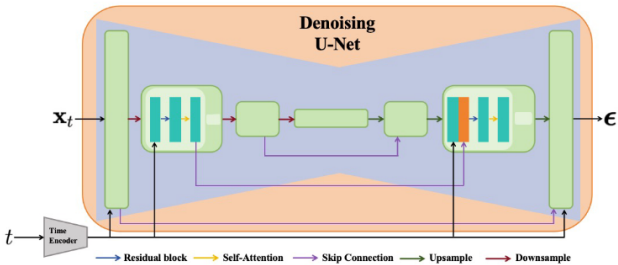


DPPM: Sampling



- To generate a sample:
  - Start from pure noise: $x_T \sim \mathcal{N}(0, I)$.
  - For $t = T$ to 1:
    - predict noise $\epsilon_\theta(x_t, t)$
    - Subtract predicted noise from the noisy image to get x_{t-1}
    - Now x_{t-1} acts as a input to the noise predictor network for the next step
    - This is repeated

DDPM: Architecture

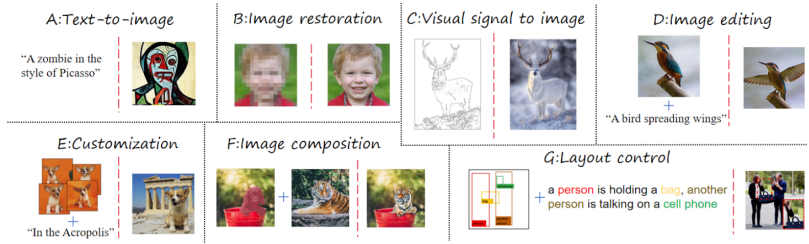


- Diffusion models are typically implemented with U-Nets (the same architecture used in image segmentation). It predicts the noise ϵ based on the given noisy input x_t and time step t.
- The timestep t is firstly converted to high-dimensional representations via a time encoder.

Conditional Diffusion Models

- The diffusion model we saw so far is unconditional, meaning it can generate samples from noise without any explicit control on what the model should generate (see image on the right).

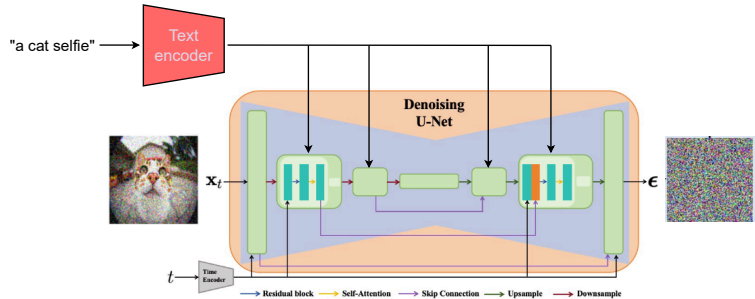
Conditional Diffusion Models



- Conditional diffusion models incorporates various kinds of information conditioning information into the diffusion process:
  - Class labels
  - Semantic maps
  - Keypoints
  - Text
  - ... many more (See paper[2] for a complete survey)

[2]Survey on conditional image synthesis

Text-to-Image Diffusion Models

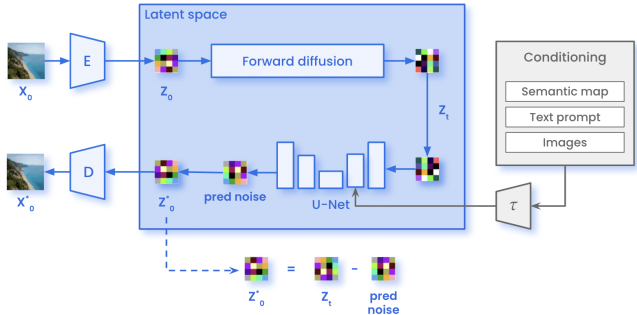


- The conditional diffusion models that incorporates text into the learning process are called text-to-image diffusion models.
- Text (or caption) is encoded through a text encoder (eg, BERT) and injected into the U-Net through cross-attention layers.
- At inference time, given pure Gaussian noise and a starting caption, the model can generate images that adheres to the input caption.

ControlNet: Sketch-to-Image Diffusion Models

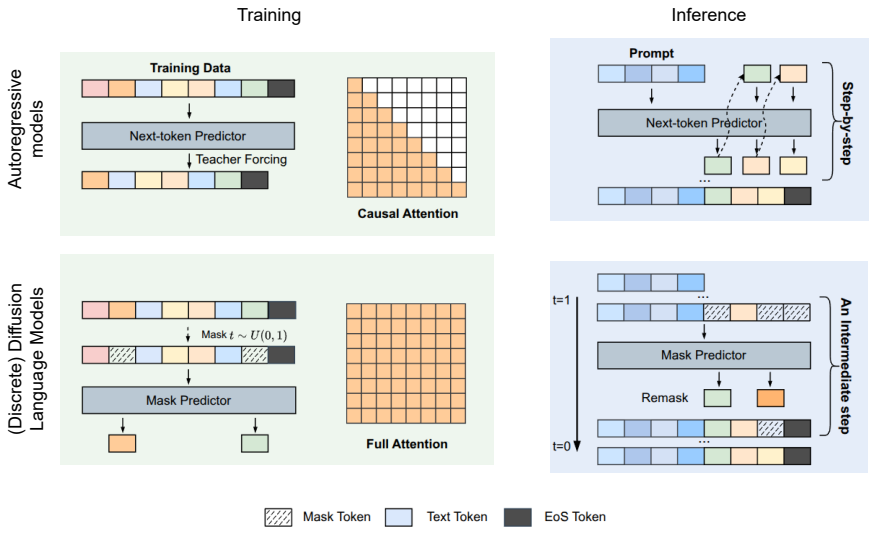
(Left) Cartoon drawing is given as an input condition to the diffusion Model and (Right) the model generates a colored version of the same sketch. Code.

Latent Diffusion Models



- **Key idea**: Perform diffusion in a compressed latent space for efficiency, then decode to high-resolution data (e.g., images).
- Works as follows:
  - Encoder takes x and compresses it into z in the latent space.
  - Apply DDPM in the latent space.
  - Decode (or reconstruct) denoised z into data space using a Decoder.

Diffusion Language Models

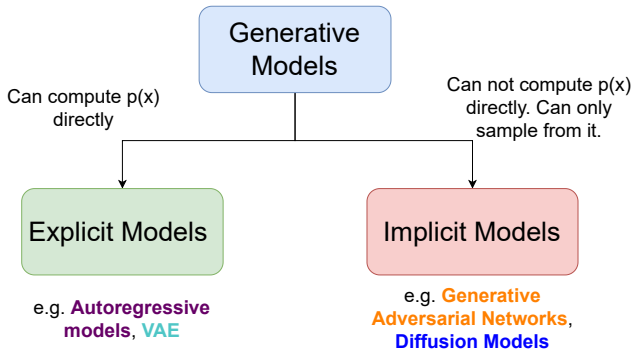


Diffusion Language Models

- Instead of predicting the next word one at a time like traditional LMs, diffusion language models[3] generate text by repeatedly denoising a corrupted sentence in parallel.
- Unlike images (Gaussian noise), language uses discrete corruption:
  - Masking tokens: [MASK]
  - Replacing with random tokens
  - Deleting or shuffling words
- During inference, at each timestep:
  - The model predicts a distribution for each position
  - Some tokens are filled in or updated

[3] see survey on diffusion LMs

Generative Models: Outro



- There are way too many generative models to cover in a lecture (or a few).
- Generative Models can be a course by itself!

References and Links

- For additional reading materials refer to the following links
  - "Variational autoencoders"[4].
  - "What are Diffusion Models?"[5].
  - "The Illustrated GPT-2"[6].
- Slide courtesy: S. Yeung, P. Abbeel, and S. Ermon.

[4] https://www.jeremyjordan.me/variational-autoencoders/
[5] https://lilianweng.github.io/posts/2021-07-11-diffusion-models/
[6] https://jalammar.github.io/illustrated-gpt2/